

WebSockets

Juegos en Red - Grado en Diseño y Desarrollo de Videojuegos

Ruben Rodríguez

ruben.rodriguez@urjc.es

URJC

Natalia Madrueño

natalia.madrueño@urjc.es

URJC

Mario Manzano

mario.manzano@urjc.es

URJC

2026-09-09



Tabla de contenidos

- [Introducción a Sockets](#)
- [WebSockets](#)
- [Negociación de Conexión](#)
- [WebSockets en Juegos](#)
- [API WebSocket en JavaScript](#)
- [Servidor WebSocket con Node.js](#)
- [Integración REST + WebSockets](#)
- [Resumen](#)



Introducción a Sockets

Concepto Básico de Socket

Socket: extremo de comunicación bidireccional

- Identificado por **IP + Puerto**
- Permite comunicación entre dos programas
- Bidireccional: ambos pueden enviar y recibir
- Simultáneo: no hay que esperar turnos

Ejemplo:

- Servidor: `212.128.240.50:10000`
- Cliente: `1XX.XXX.XXX.XX:YYYY`



Proceso de Comunicación con Sockets

Flujo típico:

1. **Servidor** escucha en puerto específico
2. **Cliente** inicia petición de conexión
3. **Servidor** acepta la conexión
4. Se establece **canal bidireccional**
5. Ambos pueden enviar/recibir datos

! Importante

No es solicitud-respuesta: ambos pueden iniciar comunicación



WebSockets

¿Qué son los WebSockets?

Protocolo de comunicación full-duplex

- Canal permanente entre cliente y servidor
- Comunicación **bidireccional** simultánea
- No necesita abrir nueva conexión cada vez
- Ideal para aplicaciones en tiempo real

Nota

Full-duplex: ambas partes pueden enviar mensajes independientemente

Funcionamiento de WebSockets

Proceso:

1. Cliente solicita conexión al servidor
2. Servidor acepta la conexión
3. Se genera **conexión permanente**
4. Procesos en ambos lados escuchan mensajes



Tip

El servidor puede enviar sin que el cliente pregunte constantemente

WebSockets como Estándar

RFC 6455 sobre socket TCP

Ventajas:

- Usa los puertos estándar **80/443** (los mismos que HTTP/HTTPS)
- Evita problemas con firewalls y proxies
- Pensado para navegadores y servidores web

Nota

Funciona sobre HTTP pero puede usar protocolo propio

Protocolos WebSocket

Dos esquemas disponibles:

- **ws://** - WebSocket en claro
 - Equivalente a **http://**
- **wss://** - WebSocket seguro (TLS)
 - Equivalente a **https://**

```
1 new WebSocket('ws://example.com/demo');  
2 new WebSocket('wss://example.com/demo');
```



Negociación de Conexión

Handshake Inicial

La negociación comienza sobre HTTP

Petición del cliente:

```
1 GET /demo HTTP/1.1
2 Host: example.com
3 Connection: Upgrade
4 Upgrade: websocket
5 Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
6 Sec-WebSocket-Version: 13
7 Origin: http://example.com
```

- **Connection: Upgrade** → queremos cambiar de protocolo
- **Upgrade: websocket** → especifica WebSocket
- **Sec-WebSocket-Key** → valor aleatorio para verificar el handshake
- **Sec-WebSocket-Version: 13** → versión del protocolo (RFC 6455)
- **Origin** → origen de la petición (seguridad)



Respuesta del Servidor

Si acepta la conexión:

```
1 HTTP/1.1 101 Switching Protocols
2 Upgrade: websocket
3 Connection: Upgrade
4 Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

- Código **101**: cambio de protocolo aceptado
- **Sec-WebSocket-Accept** → derivado de **Sec-WebSocket-Key** (demuestra que el servidor entiende WebSocket)

⚠ Importante

Después del handshake, HTTP se transforma en WebSocket

Uso Práctico

La negociación es automática

```
1 // Esto maneja todo el handshake automáticamente  
2 const connection = new WebSocket('ws://example.com/demo');
```



Tip

No necesitas implementar manualmente el handshake HTTP

WebSockets en Juegos

Ventajas para Juegos en Red

Por qué usar WebSockets:

- **Tiempo real**
 - Actualizaciones instantáneas
 - Sin demoras
- **Bidireccional**
 - Servidor puede enviar eventos
 - Sin polling constante
- **Eficiente**
 - Una conexión persistente
 - Menos recursos
- **Baja latencia**
 - Ideal para respuestas rápidas
 - Crítico en juegos competitivos



WebSockets vs REST en Juegos

Cuándo usar cada uno:

REST	WebSockets
Registro/autenticación	Movimiento de jugadores
Puntuaciones	Estado del juego
Rankings	Eventos en tiempo real
Perfiles	Chat del juego
Configuración	Notificaciones instantáneas

Nota

Ambas tecnologías se complementan en juegos modernos

API WebSocket en JavaScript

Crear Conexión

Instanciar el objeto WebSocket

```
1 const connection = new WebSocket('ws://IP:PUERTO/RUTA');
```

Componentes:

- `ws://` o `wss://` → protocolo
- `IP` → dirección o dominio del servidor
- `PUERTO` → puerto donde escucha el servidor
- `RUTA` → endpoint específico

Ejemplo:

```
1 const connection = new WebSocket('ws://127.0.0.1:8080/echo');
```



Tip

La conexión se establece automáticamente



Métodos Principales

send() - Enviar datos al servidor

```
1 // Texto simple
2 connection.send('Hola servidor');
3
4 // JSON (debe convertirse a string)
5 const data = {
6   type: 'player_move',
7   x: 150,
8   y: 200
9 };
10 connection.send(JSON.stringify(data));
```

close() - Cerrar conexión

```
1 connection.close();
```

Nota

`close()` inicia cierre ordenado de la conexión

Event Listeners - onopen

Se ejecuta cuando la conexión se abre

```
1 connection.onopen = function() {  
2   console.log('¡Conectado al servidor!');  
3  
4   // Enviar información inicial  
5   connection.send(JSON.stringify({  
6     type: 'player_join',  
7     username: 'Jugador1'  
8   }));  
9 };
```



Tip

Momento perfecto para enviar mensaje inicial

Event Listeners - onmessage

Se ejecuta al recibir mensaje del servidor

```
1 connection.onmessage = function(msg) {  
2   console.log("Mensaje recibido: " + msg.data);  
3  
4   // Si es JSON, parsearlo  
5   const data = JSON.parse(msg.data);  
6  
7   // Procesar según tipo  
8   if (data.type === 'player_position') {  
9     actualizarPosicionJugador(data.playerId, data.x, data.y);  
10  } else if (data.type === 'game_over') {  
11    mostrarPantallaFinJuego(data.winner);  
12  }  
13 };
```

⚠ Importante

El dato real está en `msg.data`

Event Listeners - onerror

Se ejecuta cuando hay un error

```
1 connection.onerror = function(error) {  
2   console.log("Error en WebSocket: ", error);  
3  
4   // Informar al usuario  
5   alert('No se pudo conectar al servidor. Verifica tu conexión.');
```

Situaciones que disparan error:

- No se puede conectar al servidor
- Problema de red
- Servidor rechaza la conexión

Event Listeners - onclose

Se ejecuta cuando la conexión se cierra

```
1 connection.onclose = function(event) {  
2   console.log('Desconectado del servidor');  
3   console.log('Código de cierre:', event.code);  
4  
5   // Notificar al usuario  
6   alert('Se ha perdido la conexión');  
7  
8   // Intentar reconectar después de 3 segundos  
9   setTimeout(function() {  
10    const nuevaConexion = new WebSocket('ws://127.0.0.1:8080/echo');  
11    // configurarListeners reasigna los manejadores onopen, onmessage,  
12    // onerror y onclose (vistos en este tema) a la nueva conexión  
13    configurarListeners(nuevaConexion);  
14  }, 3000);  
15 };
```

Razones del cierre:

- Llamamos a `connection.close()`
- El servidor cierra la conexión
- Se pierde la conexión de red



Ejemplo Completo - Cliente

```
1 // Crear conexión
2 const ws = new WebSocket('ws://127.0.0.1:8080/game');
3
4 // Configurar listeners
5 ws.onopen = function() {
6   console.log('Conectado');
7   ws.send(JSON.stringify({ type: 'join', name: 'Alice' }));
8 };
9
10 ws.onmessage = function(msg) {
11   const data = JSON.parse(msg.data);
12   console.log('Recibido:', data);
13 };
14
15 ws.onerror = function(error) {
16   console.error('Error:', error);
17 };
18
19 ws.onclose = function() {
20   console.log('Desconectado');
21 };
22
23 // Enviar movimiento
24 function moverJugador(x, y) {
25   ws.send(JSON.stringify({ type: 'move', x, y }));
```



Servidor WebSocket con Node.js

Librería ws

Implementación robusta y eficiente

Instalación:

```
1 npm init -y
2 npm install ws express
```

Configurar package.json:

```
1 {
2   "name": "websocket-server",
3   "version": "1.0.0",
4   "type": "module",
5   "dependencies": {
6     "ws": "^8.0.0",
7     "express": "^4.18.0"
8   }
9 }
```



Servidor Básico

```
1 import express from 'express';
2 import { WebSocketServer } from 'ws';
3 import { createServer } from 'http';
4
5 const app = express();
6 const server = createServer(app);
7
8 // Servir archivos estáticos
9 app.use(express.static('public'));
10
11 // Crear servidor WebSocket
12 const wss = new WebSocketServer({ server });
13
14 // Iniciar servidor
15 const PORT = 8080;
16 server.listen(PORT, () => {
17   console.log(`Servidor en http://localhost:${PORT}`);
18 });
```



Estructura del Servidor

Componentes:

1. **Servidor HTTP** (`createServer`)
 - Sirve archivos estáticos
 - Comparte puerto con WebSocket
2. **Express** para archivos estáticos
 - Carpeta `public` para cliente HTML/CSS/JS
3. **WebSocketServer** asociado al servidor HTTP
 - Ambos protocolos en el mismo puerto



Evento connection

Se dispara cuando un cliente se conecta

```
1 wss.on('connection', (ws) => {  
2   console.log('Nuevo cliente conectado');  
3  
4   // ws representa la conexión con este cliente específico  
5 });
```

Nota

Cada cliente tiene su propio objeto `ws`

Recibir Mensajes del Cliente

```
1 wss.on('connection', (ws) => {  
2   console.log('Nuevo cliente conectado');  
3  
4   ws.on('message', (message) => {  
5     console.log('Mensaje recibido:', message.toString());  
6  
7     // Si es JSON, parsearlo  
8     const data = JSON.parse(message.toString());  
9     console.log('Nombre:', data.nombre);  
10    console.log('Mensaje:', data.mensaje);  
11  });  
12 });
```

⚠ Importante

Los mensajes llegan como `Buffer`, usar `.toString()`

Enviar Mensajes al Cliente

A un cliente específico:

```
1 ws.on('message', (message) => {  
2   const received = message.toString();  
3  
4   // Enviar respuesta  
5   ws.send(`Echo: ${received}`);  
6 });
```

Enviar JSON:

```
1 ws.on('message', (message) => {  
2   const respuesta = {  
3     tipo: 'confirmacion',  
4     timestamp: new Date().toISOString(),  
5     mensaje: 'Recibido correctamente'  
6   };  
7  
8   ws.send(JSON.stringify(respuesta));  
9 });
```



Eventos close y error

Limpiar recursos al desconectar:

```
1 wss.on('connection', (ws) => {
2   console.log('Cliente conectado');
3
4   ws.on('close', () => {
5     console.log('Cliente desconectado');
6     // Limpiar datos asociados a este cliente
7   });
8
9   ws.on('error', (error) => {
10    console.error('Error en la conexión:', error);
11  });
12
13  ws.on('message', (message) => {
14    // Manejar mensajes
15  });
16 });
```



Ejemplo: Servidor Echo

Devuelve cualquier mensaje recibido

```
1 import express from 'express';
2 import { WebSocketServer } from 'ws';
3 import { createServer } from 'http';
4
5 const app = express();
6 const server = createServer(app);
7
8 app.use(express.static('public'));
9
10 const wss = new WebSocketServer({ server, path: '/echo' });
11
12 wss.on('connection', (ws) => {
13   console.log('Nuevo cliente');
14
15   ws.on('message', (message) => {
16     const data = message.toString();
17     console.log('Recibido:', data);
18
19     // Echo: devolver al cliente
20     ws.send(`Echo: ${data}`);
21   });
22
23   ws.on('close', () => console.log('Cliente desconectado'));
24   ws.on('error', (error) => console.error('Error:', error));
25 });
```



Broadcast: Enviar a Todos

Enviar mensaje a todos los clientes conectados

```
1 wss.on('connection', (ws) => {  
2   ws.on('message', (message) => {  
3     const data = JSON.parse(message.toString());  
4  
5     // Enviar a todos los clientes  
6     wss.clients.forEach((client) => {  
7       if (client.readyState === WebSocket.OPEN) {  
8         client.send(JSON.stringify(data));  
9       }  
10    });  
11  });  
12 });
```



Tip

Verificar `readyState === WebSocket.OPEN` antes de enviar (importando `WebSocket` de 'ws')

Ejemplo: Servidor de Chat

```
1 import express from 'express';
2 import { WebSocketServer, WebSocket } from 'ws';
3 import { createServer } from 'http';
4
5 const app = express();
6 const server = createServer(app);
7 app.use(express.static('public'));
8
9 const wss = new WebSocketServer({ server });
10
11 wss.on('connection', (ws) => {
12   console.log('Cliente conectado al chat');
13
14   ws.on('message', (message) => {
15     try {
16       const datos = JSON.parse(message.toString());
17       console.log(`${datos.nombre}: ${datos.mensaje}`);
18
19       const respuesta = {
20         nombre: datos.nombre,
21         mensaje: datos.mensaje,
22         timestamp: new Date().toISOString()
23       };
24
25       // Broadcast a todos
```



Gestionar Estado de Clientes

Asociar datos a cada conexión

```
1  const clientes = new Map();
2  let nextClientId = 1;
3
4  wss.on('connection', (ws) => {
5    // Asignar ID único
6    const clientId = nextClientId++;
7    clientes.set(ws, { id: clientId, nombre: null });
8
9    console.log(`Cliente ${clientId} conectado`);
10
11    ws.on('message', (message) => {
12      const datos = JSON.parse(message.toString());
13
14      // Guardar nombre del cliente
15      const clienteInfo = clientes.get(ws);
16      clienteInfo.nombre = datos.nombre;
17
18      // Procesar mensaje...
19    });
20
21    ws.on('close', () => {
22      const clienteInfo = clientes.get(ws);
23      console.log(`Cliente ${clienteInfo.id} desconectado`);
24      clientes.delete(ws);
25    });
```



Integración REST + WebSockets

Arquitectura Híbrida

Combinar lo mejor de ambos mundos

```
1 import express from 'express';
2 import { WebSocketServer } from 'ws';
3 import { createServer } from 'http';
4
5 const app = express();
6 const server = createServer(app);
7
8 app.use(express.json());
9 app.use(express.static('public'));
10
11 // Rutas REST
12 app.get('/api/status', (req, res) => {
13   res.json({
14     clientesConectados: wss.clients.size,
15     estado: 'activo'
16   });
17 });
18
19 // Servidor WebSocket
20 const wss = new WebSocketServer({ server });
21
22 wss.on('connection', (ws) => {
23   // Manejar conexiones WebSocket
24 });
25
```



División de Responsabilidades

REST para:

- Autenticación
- Registro de usuarios
- Consulta de rankings
- Subida de archivos
- Operaciones puntuales

WebSocket para:

- Movimiento de jugadores
- Estado del juego en tiempo real
- Notificaciones instantáneas
- Chat del juego
- Eventos en vivo

! Importante

Ambos protocolos comparten servidor y puerto

Ventajas de Arquitectura Híbrida

Lo mejor de ambos mundos:

- **Simplicidad REST** para operaciones CRUD
- **Tiempo real WebSocket** para interactividad
- **Mismo servidor** simplifica despliegue
- **Mismo puerto** evita problemas de firewall
- **Código organizado** por tipo de comunicación



Resumen

Conceptos Clave de WebSockets

Fundamentos:

- Socket: extremo de comunicación bidireccional
- WebSocket: protocolo full-duplex sobre TCP
- Conexión permanente entre cliente y servidor
- Handshake inicial sobre HTTP (código 101)
- Protocolos `ws://` y `wss://`

Ventajas:

- Comunicación en tiempo real
- Bidireccional y simultánea
- Eficiente (una conexión persistente)
- Baja latencia para juegos



API JavaScript

Objeto WebSocket:

```
1 const ws = new WebSocket('ws://host:port/path');
```

Métodos:

- `send(data)` - enviar al servidor
- `close()` - cerrar conexión

Event Listeners:

- `onopen` - conexión establecida
- `onmessage` - mensaje recibido
- `onerror` - error en conexión
- `onclose` - conexión cerrada



Servidor Node.js

Librería ws:

```
1 import { WebSocketServer, WebSocket } from 'ws';  
2 const wss = new WebSocketServer({ server });
```

Eventos del servidor:

- `connection` - nuevo cliente conectado
- `message` - mensaje del cliente
- `close` - cliente desconectado
- `error` - error en conexión

Broadcast:

```
1 wss.clients.forEach(client => {  
2   if (client.readyState === WebSocket.OPEN) {  
3     client.send(data);  
4   }  
5 });
```



REST vs WebSockets

Cuándo usar cada tecnología:

Criterio	REST	WebSockets
Frecuencia	Ocasional	Continua
Latencia	Aceptable	Crítica
Dirección	Cliente inicia	Ambos inician
Casos de uso	CRUD, auth	Tiempo real, chat



Tip

En juegos modernos: ambas tecnologías se complementan